Московский государственный университет имени М.В. Ломоносова Факультет вычислительной математики и кибернетики Кафедра автоматизации систем вычислительных комплексов

Ефанов Михаил Михайлович

Алгоритм управления перегрузкой на основе пропускной способности и задержки в условиях применения пакетной балансировки

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Научный руководитель:
д.ф.-м.н., профессор, чл.-корр. РАН Смелянский Руслан Леонидович

Научный консультант:
к.ф.-м.н.,ассистент Степанов Евгений Павлович

Москва, 2026

Аннотация

Работа посвящена разработке алгоритма управления перегрузкой на основе пропускной способности и задержки (BBR), адаптированного для работы в условиях пакетной балансировки. Пакетная балансировка значительно повышает производительность и надежность сетевых сервисов за счет горизонтального масштабирования, однако классические алгоритмы управления перегрузкой не адаптированы для совместной работы с ней.

Основные проблемы, возникающие при использовании пакетной балансировки, включают переупорядочивание пакетов и перегрузку каналов, которая приводит к ложным сигналам перегрузки и снижению скорости передачи данных. В работе проведен обзор существующих решений, направленных на повышение устойчивости алгоритмов управления перегрузкой к переупорядочиванию пакетов и перегрузке каналов: TCP-NCR, Lazy TCP (LTCP) и MSwift.

На основе сравнительного анализа для модификации алгоритма BBR выбран подход, используемый в MSwift, который обеспечивает устойчивость как к переупорядочиванию пакетов, так и к перегрузке каналов. Разработан стенд для моделирования пакетной балансировки в системе ns3, позволяющий исследовать поведение алгоритмов в условиях асимметрии каналов.

Оглавление

Введение . . . 3
1. Постановка задачи . . . 5
2. Обзор существующих решений . . . 5
   2.1. Основные проблемы при пакетной балансировке . . . 5
      2.1.1. Переупорядочивание пакетов . . . 5
      2.1.2. Деградация отдельных путей . . . 6
   2.2. BBR как базовый алгоритм управления перегрузкой . . . 6
   2.3. TCP-NCR: динамический порог реакции на duplicate ACK . . . 9
   2.4. Lazy TCP (LTCP): “ленивый триггер” и “упрямое окно” . . . 10
   2.5. MSwift: устойчивая к пакетной балансировке агрегация сигналов (медиана) . . . 11
   2.6. Выводы по обзору и выбор направления для модификации BBR . . . 12
3. Исследование и решение задачи . . . 13
Литература . . . 14

Введение

Современные компьютерные сети характеризуются высокой динамикой нагрузки, неоднородностью каналов и всё более широким применением многопутевой передачи. На практике трафик одного транспортного соединения может проходить по нескольким параллельным маршрутам — как из-за особенностей маршрутизации и инженерии трафика в сети, так и из-за применения механизмов балансировки нагрузки. При этом балансировка может выполняться не только по потокам, но и на уровне отдельных пакетов (per-packet load balancing, packet spraying), когда последовательные пакеты одного и того же соединения целенаправленно распределяются по разным путям. Такая стратегия потенциально повышает использование доступной пропускной способности совокупности путей и улучшает устойчивость передачи к локальным перегрузкам и отказам, но одновременно меняет свойства наблюдаемых транспортом сигналов состояния сети [1].

Классические алгоритмы управления перегрузкой TCP исторически опираются на интерпретацию потери пакета (loss-based) как основного признака перегрузки. Эта эвристика была разумной в условиях ограниченных буферов и относительно простых путей, однако в современных сетях приводит к систематическим проблемам. При больших буферах на узких местах loss-based управление перегрузкой склонно формировать устойчивые очереди и увеличивать задержки, а при потерях, не связанных с перегрузкой, — неоправданно снижать скорость, недоиспользуя доступную ёмкость канала. На этом фоне получили развитие подходы, основанные не на потерях, а на моделировании физических ограничений пути: пропускной способности узкого места (BtlBw) и задержки распространения (RTprop). Алгоритм BBR является представителем этого направления: он оценивает BtlBw и RTprop, стремится поддерживать объём данных «в полёте» (отправленные пакеты на, которые еще не получен ACK) около BDP = BtlBw × RTprop, а темп передачи задаёт через задержку между отправкой следующего пакета (pacing). Такой подход позволяет одновременно добиваться высокой полезной пропускной способности и удерживать задержки на низком уровне, что подтверждается опытом крупномасштабного применения в реальных сервисах и на разнообразных типах сетей, включая мобильные [2].

Однако применение многопутевой передачи и особенно пакетной балансировки усложняет задачу управления перегрузкой. Транспортный протокол перестаёт наблюдать «один путь» с единственными характеристиками задержки и пропускной способности; вместо этого он получает поток обратной связи, являющийся смесью нескольких путей, которые могут различаться по задержкам, текущим очередям и доступной скорости. Даже если изначально пути близки по параметрам, в реальной сети неизбежны временные асимметрии: локальные перегрузки, изменения маршрута, сбои и деградации отдельных каналов, приводящие к увеличению

задержки или потере части пакетов на одном из путей. В таких условиях пакеты могут приходить не по порядку, что порождает дубликатные ACK и другие аномальные наблюдения, которые традиционный TCP склонен трактовать как перегрузку. Это приводит к неоправданному снижению скорости передачи и деградации производительности — иногда до состояния, которое можно описать как коллапс пропускной способности при наличии даже небольшой доли перегруженных путей из общего числа путей, между которыми реализована балансировка пакетов [3].

Проблема устойчивости TCP к переупорядочению пакетов рассмотрена в [4]: в этой работе введено понятие «устойчивости к неперегрузочным событиям» (Non-Congestion Robustness) и предложен подход, в котором порог срабатывания быстрой ретрансляции по duplicate ACK не фиксирован, а динамически увеличивается в зависимости от состояния соединения. Это позволяет лучше отличать истинную потерю от переупорядочения и уменьшает число избыточных повторных передач, хотя вводит компромисс в виде замедления реакции на реальную перегрузку. В работе [5] предложены более специализированные механизмы: Lazy TCP откладывает реакцию на dupACK («lazy trigger») и по-разному реагирует на единичные и последовательные потери, стремясь не снижать скорость из-за событий, которые с высокой вероятностью не являются перегрузкой.

Однако устойчивости только к переупорядочиванию пакетов часто недостаточно. Современные алгоритмы управления перегрузкой используют разные сигналы, например задержку (delay-based). В условиях пакетной балансировки эти сигналы начинают поступать как смесь измерений по разным путям и могут содержать выбросы из-за перегруженного или более длинного пути. Показано, что для широкого класса алгоритмов, использующих AIMD-подобные принципы, при наличии перегруженного пути с вероятностью попадания на него q пропускная способность может ухудшаться по закону порядка $\Theta(1/\sqrt{q})$; причём это проявляется для различных CCA и сохраняется даже у вариантов, устойчивых к переупорядочению [3]. В качестве более общего принципа предложено применять устойчивую к подобным выбросам агрегацию многопутевых сигналов, например использовать медиану по окну недавних измерений вместо реакции на «последнее» наблюдение [3].

Таким образом, актуальность темы данной дипломной работы определяется следующими обстоятельствами:

- распространённостью и практической значимостью алгоритмов управления перегрузкой, ориентированных на пропускную способность и задержку, таких как BBR, позволяющих приблизиться к высокой утилизации канала при малых очередях [2];

- растущей ролью пакетной балансировки и других форм многопутевой доставки, которые могут повышать использование доступной суммарной ёмкости, но провоцируют переупорядочение и делают транспортные измерения смешанными по нескольким путям [1];
- наличием фундаментальных эффектов деградации производительности CCA в многопутевом per-packet сценарии и необходимостью более робастной обработки сигналов перегрузки, чем простая защита от dupACK [3];
- существованием частичных решений (динамический порог dupACK, «ленивые» реакции), которые полезны, но не закрывают полностью задачу устойчивого управления при многопутевой смешанной обратной связи [3–5]

1. Постановка задачи

Целью дипломной работы является повышение скорости транспортного соединения при сохранении низкой и предсказуемой задержки за счёт разработки модификации алгоритма BBR, адаптированной к работе в условиях пакетной балансировки.

Для достижения поставленной цели необходимо решить следующие задачи:

- Провести обзор существующих алгоритмов управления перегрузкой, направленных на использование совместно с пакетной балансировкой и выявить причины уменьшения производительности CCA в условиях применения пакетной балансировки.
- Разработать модификации алгоритма BBR.
- Провести сравнительный анализа базового и модифицированного BBR.

2. Обзор существующих решений

2.1. Основные проблемы при пакетной балансировке

2.1.1. Переупорядочивание пакетов

При распылении пакетов по нескольким маршрутам пакеты одного соединения могут приходить к получателю не по порядку. Получатель в этом случае обычно отвечает **дубликатными ACK** (dupACK) — это ACK, который повторно подтверждает один и тот же номер байта, потому что следующий байт (или сегмент) ещё не получен. Отправитель может ошибочно принять серию dupACK за потерю и запустить механизмы восстановления, что снижает скорость передачи, хотя перегрузки могло не быть [1].

Будем считать транспортный агент **устойчивым к переупорядочиванию**, если переупорядочивание само по себе не приводит к уменьшению скорости передачи (то есть не запускает лишнее снижение скорости только из-за порядка прихода пакетов) [5].

2.1.2. Деградация отдельных путей

В per-packet сценарии достаточно, чтобы часть путей временно стала хуже (меньше пропускная способность, выше задержка или больше потерь), чтобы измерения на стороне транспорта начали содержать **выбросы** — редкие «плохие» измерения, нехарактерные для большинства путей. Если алгоритм управления перегрузкой реагирует на такие выбросы как на общую перегрузку всего соединения, он снижает скорость непропорционально сильно и недоиспользует суммарную емкость.

Для широкого класса AIMD-подобных алгоритмов показан эффект деградации при вероятности попадания пакета на перегруженный путь `q`: пропускная способность может ухудшаться порядка `Θ(1/√q)` [3]. Важно, что это относится не только к классическому TCP, но и к современным CCA, в том числе использующим задержку, и может сохраняться даже при добавлении устойчивости к переупорядочиванию [3].

Будем считать транспортный агент **устойчивым к деградации отдельных путей**, если наличие небольшой доли «плохих» путей не приводит к заметному уменьшению скорости на всём соединении (в идеале — скорость падает только настолько, насколько неизбежно из-за реально потерянных пакетов на плохих путях) [3].

2.2. BBR как базовый алгоритм управления перегрузкой

Алгоритм BBR (Bottleneck Bandwidth and Round-trip propagation time) относится к классу model-based алгоритмов управления перегрузкой: его работа основана не на прямой реакции на потери пакетов, а на построении явной модели пути передачи. В основе BBR лежит идея, восходящая к результатам Леонарда Клейнрока [6]: скорость отправки пакетов в сеть должна обеспечивать максимально возможную пропускную способность при минимально необходимом уровне очередей в сети.

В рамках этой модели BBR непрерывно оценивает два ключевых параметра пути:

- минимальную задержку распространения `RTprop`, приближаемую наблюдаемым минимумом времени обхода пакета туда и обратно;
- пропускную способность узкого места `BtlBw`, оцениваемую по скорости доставки данных получателю.

Идеальный объём данных в сети при этом определяется произведением пропускной способности на задержку распространения, то есть bandwidth-delay product:

$$\mathrm{BDP} = \mathrm{BtlBw} \cdot \mathrm{RTprop}$$

Величина `BDP` задаёт естественную целевую оценку числа данных «в полёте» (`inflight` - отправленные пакеты, на которые еще не был получен ACK), необходимого для насыщения канала без формирования избыточной очереди.

Соответственно, BBR управляет соединением с помощью двух взаимосвязанных механизмов:

- регулирования темпа отправки пакетов (`pacing`);
- ограничения окна перегрузки `cwnd`, которое ограничивает максимально допустимый объём неподтверждённых данных.

Темп передачи выбирается пропорционально текущей оценке пропускной способности узкого места:

$$\text{pacing_rate} = \text{pacing_gain} \cdot \text{BtlBw}$$

Здесь `pacing_gain` — коэффициент, зависящий от текущего состояния автомата BBR и определяющий, должен ли отправитель ускорять передачу, поддерживать стационарный режим или временно снижать интенсивность для дренирования очереди.

Оценка `BtlBw` формируется по наблюдаемой скорости доставки данных. В базовой версии BBR она вычисляется как максимум скорости передачи (`delivery rate`) на скользящем интервале длиной порядка десяти раундов передачи:

$$\text{BtlBw} = \max_{t \in W} \text{delivery_rate}(t), \quad |W| \approx 10 \cdot \text{RTprop}$$

Тем самым BBR стремится выделить устойчивую верхнюю оценку доступной пропускной способности, подавляя кратковременные просадки. Оценка `RTprop`, напротив, выбирается как минимум RTT на достаточно длинном временном окне, поскольку именно минимальное наблюдаемое значение наилучшим образом соответствует задержке распространения без влияния очередей:

$$\text{RTprop} = \min_{t \in W_r} \text{RTT}(t)$$

Таким образом, базовая логика BBR может быть интерпретирована как поддержание условия

$$\text{inflight} \approx \text{BDP}$$

то есть такого режима, при котором соединение заполняет канал, но по возможности не создаёт устойчивой очереди в узком месте.

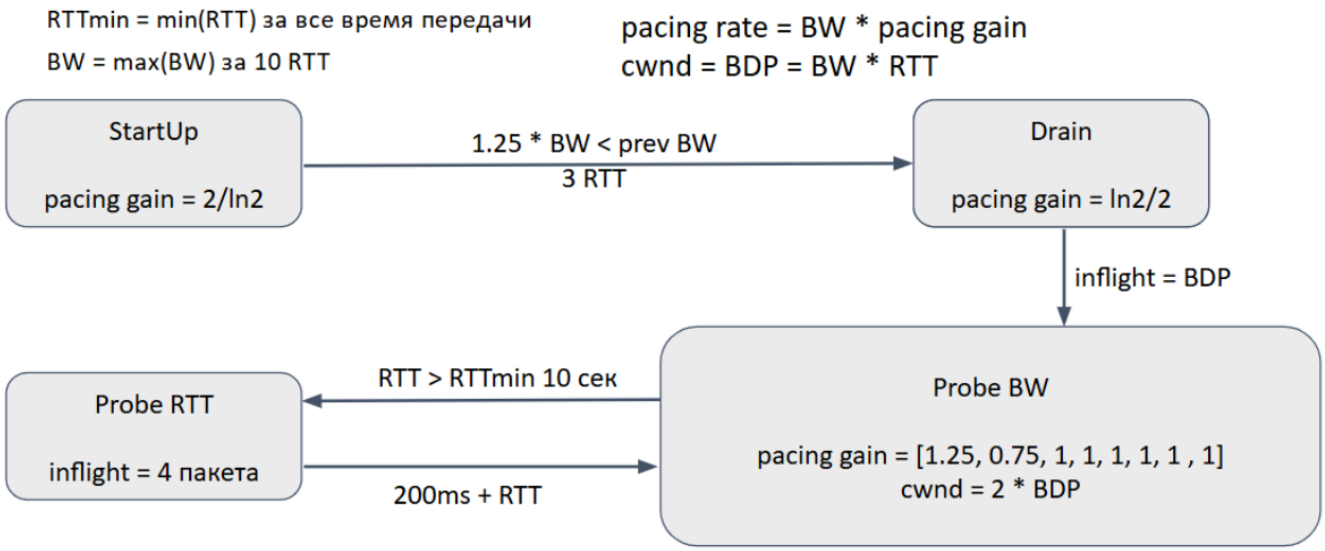

Рисунок 1: Схема автомата BBR.

Функционирование BBR удобно описывать как работу конечного автомата из четырёх основных состояний [2].

Состояние StartUp. На начальном этапе отправитель стремится быстро приблизиться к доступной пропускной способности пути. Для этого используется повышающий коэффициент темпа передачи

$$\text{pacing_gain} = \frac{2}{\ln 2}$$

что соответствует экспоненциальному наращиванию интенсивности отправки до тех пор, пока оценки скорости доставки продолжают заметно расти. Переход из состояния StartUp выполняется тогда, когда прирост оценки `BtlBw` перестаёт быть существенным в течение нескольких последовательных раундов передачи; в реализации BBR это интерпретируется как достижение насыщения узкого места.

Состояние Drain. После агрессивного заполнения канала, характерного для StartUp, в сети обычно формируется избыточная очередь. Поэтому на этапе Drain отправитель временно снижает темп передачи, чтобы удалить накопившиеся данные из буфера и вернуть соединение к режиму, близкому к целевому `BDP`. Для этого используется коэффициент

$$\text{pacing_gain} = \frac{\ln 2}{2}$$

Переход к следующему состоянию происходит, когда объём данных в сети становится не больше целевого значения, то есть выполняется условие

$$\text{inflight} \leq \text{BDP}$$

Состояние ProbeBW. Это основное рабочее состояние алгоритма, в котором соединение проводит большую часть времени. Его задача состоит в периодическом зондировании доступной пропускной способности: отправитель чередует фазы умеренного ускорения, умеренного замедления и нейтральной передачи. В классическом варианте BBR используется циклическая последовательность коэффициентов

$$(1.25, 0.75, 1, 1, 1, 1, 1, 1)$$

с переключением между фазами примерно один раз за RTT. Фаза с коэффициентом больше единицы позволяет проверить, не увеличилась ли доступная пропускная способность, а фаза с коэффициентом меньше единицы способствует уменьшению возможной очереди. В этом состоянии окно перегрузки обычно выбирается с запасом относительно `BDP` (`cwnd = 2 dot BDP`), чтобы pacing оставался основным механизмом управления скоростью, а `cwnd` не ограничивал передачу преждевременно.

Состояние ProbeRTT. Поскольку минимальная задержка распространения может со временем устаревать из-за постоянного присутствия очереди, BBR периодически переходит в режим переоценки `RTprop`. Для этого объём данных в сети искусственно уменьшается, а окно перегрузки сокращается до малого значения (в классической реализации — до четырёх сегментов), чтобы дать очередям возможность полностью опустеть. После короткого интервала наблюдения, достаточного для измерения нового минимального RTT, автомат возвращается в состояние ProbeBW [2].

2.3. TCP-NCR: динамический порог реакции на duplicate ACK

TCP-NCR (Non-Congestion Robustness) был предложен как модификация классического TCP, в котором запуск **fast retransmit** и последующее уменьшение окна перегрузки тесно связаны с получением серии duplicate ACK. Основная идея метода состоит в том, чтобы сделать порог срабатывания не фиксированным (`3 dupACK`), а зависящим от текущего состояния соединения: чем больше данных находится “в полёте”, тем больший уровень переупорядочивания допускается без немедленного объявления потери [4].

Порог задаётся выражением

$$\text{DupThresh} = \max\left(\text{LT_F} \cdot \frac{\text{InFlightSize}}{\text{SMSS}}, 3\right)$$

где `InFlightSize` — текущий объём неподтверждённых данных, `SMSS` — размер сегмента, а `LT_F` — параметр настройки алгоритма. Дополнительно TCP-NCR

использует **Extended Limited Transmit**: при поступлении duplicate ACK отправитель продолжает осторожно вводить в сеть новые сегменты, чтобы сохранить самотактирование ACK-потоком и не остановить передачу до накопления достаточного числа подтверждений.

Такой подход действительно полезен для loss-based TCP, поскольку в нём duplicate ACK являются одним из центральных сигналов, определяющих и факт потери, и момент снижения `cwnd`. Однако для BBR этот механизм концептуально плохо подходит. Как показано в разделе [BBR как базовый алгоритм управления перегрузкой](diplom.typ), BBR не использует правило вида «получили `k` duplicate ACK — уменьшаем окно и скорость». Его основное управление строится вокруг модели пути, включающей оценки `BtlBw` и `RTprop`, а темп передачи задаётся через `pacing_rate`, а не через непосредственную реакцию на число duplicate ACK. Следовательно, динамическая настройка `DupThresh` затрагивает лишь периферийный механизм обнаружения потерь, но не влияет на главный источник ошибок BBR при packet spraying — искажение измерений скорости доставки и RTT из-за смешения наблюдений от нескольких путей. Поэтому TCP-NCR можно рассматривать лишь как локальную защиту от ложного fast retransmit, но не как подходящую основу для модификации BBR в условиях пакетной балансировки.

2.4. Lazy TCP (LTCP): “ленивый триггер” и “упрямое окно”

Lazy TCP также разрабатывался для сценария packet spraying, но, как и TCP-NCR, остаётся привязанным к классической TCP-логике управления через потери и окно перегрузки. В работе [5] предложены два основных механизма.

Первый механизм — **lazy trigger**: отправитель не реагирует немедленно на серию duplicate ACK, а откладывает запуск восстановления, чтобы дать переупорядоченным пакетам возможность прийти без ложного уменьшения скорости. Второй механизм — **stubborn window**: алгоритм различает **дискретные** и **последовательные** потери. Если по информации `SACK` видно, что потери единичны и не образуют длинную непрерывную последовательность, LTCP трактует их как следствие деградации отдельного пути и не уменьшает `cwnd`; если же потери выглядят как серия подряд идущих сегментов, они интерпретируются как признак перегрузки, и окно уменьшается. Для различения типов потерь используется `SACK` (Selective ACK) — это опция TCP, в которой получатель сообщает, какие блоки данных он уже получил; по ней отправитель может понять, какие сегменты отсутствуют. Критерий «последовательности» задаётся порогом `k`: если потеряно больше `k` пакетов подряд, потери считаются связанными с перегрузкой (successive loss), иначе — дискретными. `k` трактуется как ожидаемое количество соседних пакетов, которые могли попасть на один перегруженный канал при распылении.

Для классического TCP такая логика естественна, потому что именно `cwnd` является главным регулятором интенсивности передачи. Но в BBR окно перегрузки играет вспомогательную роль: оно ограничивает верхнюю границу `inflight`, тогда как фактическая скорость передачи определяется `pacing_rate`, вычисляемым из оценки `BtlBw`. Поэтому перенос идеи **stubborn window** в BBR не даёт ожидаемого эффекта. Даже если не уменьшать `cwnd` при «дискретных» потерях, BBR всё равно будет принимать решения на основе наблюдаемой скорости доставки и RTT. Кроме того, классификация потерь на дискретные и последовательные в LTCP опирается на предположение о количестве подряд идущих пакетов, попавших в один канал при распылении; в реальных сетях эту величину трудно оценить заранее и она может меняться во времени. По этим причинам LTCP также не подходит как направление модификации BBR.

2.5. MSwift: устойчивая к пакетной балансировке агрегация сигналов (медиана)

На первый взгляд MSwift выглядит существенно ближе к потребностям BBR, чем TCP-NCR и LTCP, поскольку в работе [3] рассматривается уже не только проблема duplicate ACK, но и общая задача робастной обработки сигналов перегрузки при packet spraying. В MSwift наряду с механизмами LTCP используется медианная агрегация задержки: вместо реакции на отдельное измерение RTT алгоритм вычисляет медиану по окну последних наблюдений,

$$N = \max(\min(\alpha \cdot \mathrm{cwnd}, K), 3)$$

, и затем принимает решение об уменьшении окна, если медианная задержка превышает заданный порог.

Однако и этот подход не переносится на BBR напрямую. Причина состоит в том, что MSwift остаётся delay-based вариантом семейства Swift/AIMD, где управляющее воздействие формулируется как реакция на агрегированную задержку: если задержка слишком велика, `cwnd` уменьшается, обычно мультипликативно. BBR устроен принципиально иначе. Во-первых, он не использует пороговую схему вида «RTT выше порога — уменьшить окно вдвое»; во-вторых, его оценка `RTprop` по смыслу является не типичным, а минимальным RTT на длинном окне, то есть попыткой выделить задержку распространения без очереди. Замена этой оценки на медиану разрушила бы сам физический смысл параметра `RTprop`: медиана отражает типичное состояние смеси путей и очередей, а BBR требуется нижняя опорная оценка, близкая к базовой задержке.

Аналогичная проблема возникает и для оценки пропускной способности. В BBR параметр `BtlBw` строится как максимум наблюдаемой `delivery_rate` на скользящем окне, поскольку алгоритм стремится не усреднить скорость, а выделить устойчивую

верхнюю оценку доступной пропускной способности узкого места. Робастная агрегация по медиане, полезная для подавления выбросов в delay-based алгоритме, здесь меняет саму цель оценивания: вместо оценки потенциально достижимой скорости получится «типичная» скорость смеси путей, что приведёт к систематическому занижению модели и недоиспользованию канала. Следовательно, MSwift важен как указание на сам факт необходимости робастной обработки многопутевых измерений, но его конкретный механизм — медиана RTT с последующим уменьшением `cwnd` — несовместим с базовыми принципами работы BBR.

2.6. Выводы по обзору и выбор направления для модификации BBR

	TCP-NCR	LTCP	MSwift
Устойчивость к переупорядочиванию пакетов	Да	Да	Да
Устойчивость к перегрузке каналов	Нет	Да	Да
Независимость от стратегии балансировки	Да	Нет	Нет, но можно настроить оптимально для большинства случаев
Отсутствие избыточных потерь при реальной перегрузке	Нет	Нет	Нет
Применимость к BBR	Нет	Нет	Нет

Рисунок 1: Сравнение подходов TCP-NCR, LTCP и MSwift

Рассмотренные подходы решают разные частные проблемы packet spraying, однако ни один из них не может быть непосредственно использован как модификация BBR. TCP-NCR и LTCP ориентированы на классический TCP, в котором ключевыми управляющими событиями являются duplicate ACK, fast retransmit и изменение `cwnd` после потерь. Для BBR такие механизмы вторичны: его поведение определяется моделью пути, оценками `BtlBw` и `RTprop`, а также управлением через pacing. Поэтому улучшение обработки duplicate ACK само по себе не устраняет главную причину деградации BBR при пакетной балансировке — искажение измерений, из которых строится модель.

MSwift поднимается на более общий уровень и показывает, что при многопутевой доставке требуется робастная обработка сигналов. Тем не менее его конкретная реализация также не подходит для BBR, поскольку основана на медианной оценке RTT и пороговом уменьшении `cwnd`, тогда как BBR нуждается в оценках другого типа: минимальной задержки распространения и верхней оценки пропускной способности. Прямая замена этих оценок медианой или перенос AIMD-реакции на задержку привели бы не к адаптации BBR, а к фактической подмене его модели другим алгоритмом.

Следовательно, направление модификации BBR должно опираться не на прямое заимствование TCP-NCR, LTCP или MSwift, а на сохранение базовой модели BBR при разработке специальных, совместимых с ней методов интерпретации многопутевых измерений. Иначе говоря, задача состоит в том, чтобы сделать оценки `RTprop`, `BtlBw` и связанных с ними величин устойчивыми к переупорядочиванию и смешению сигналов от нескольких путей [2–5].

3. Исследование и решение задачи

Литература

1. Dixit A. et al. On the Impact of Packet Spraying in Data Center Networks. 2013. P. 9.
2. Cardwell N. et al. BBR: Congestion-Based Congestion Control // Queue. 2016. P. 34.
3. Gerstein B., Silberstein M., Keslassy I. Making congestion control robust to per-packet load balancing in datacenters. 2025. P. 10.
4. Bhandarkar S. et al. Improving the Robustness of TCP to Non-Congestion Events. 2006. P. 18.
5. Zhang J., Zhang D., Huang K. Improving datacenter throughput and robustness with Lazy TCP over packet spraying // Computer Communications. 2015. P. 11.
6. Kleinrock L. Power and deterministic rules of thumb for probabilistic problems in computer communications // IEEE. 1979. P. 10.